6_2

September 20, 2026

1 Bài 6.2 — Cài đặt giải thuật Apriori

Yêu cầu: cài đặt Apriori bằng Python, chạy thử với dữ liệu Market-Basket ở slide 3 của buổi lý thuyết, chọn `minSup` / `minConf` phù hợp, xuất ra các tập phổ biến và luật kết hợp kèm support / confidence.

Nội dung: 1. Dữ liệu Market-Basket (slide 3) 2. Cài đặt Apriori từ đầu (không dùng thư viện) 3. Chọn ngưỡng `minSup`, `minConf` 4. Kết quả: các tập phổ biến F_k 5. Kết quả: các luật kết hợp 6. Kiểm chứng lại bằng package FIM (PyFIM)

[1]:
```
from itertools import combinations
import pandas as pd
```

1.1 1. Dữ liệu Market-Basket (slide 3)

TID	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Tập item: `{Bread, Milk, Diaper, Beer, Eggs, Coke}`

[2]:
```
transactions = {
    1: {"Bread", "Milk"},
    2: {"Bread", "Diaper", "Beer", "Eggs"},
    3: {"Milk", "Diaper", "Beer", "Coke"},
    4: {"Bread", "Milk", "Diaper", "Beer"},
    5: {"Bread", "Milk", "Diaper", "Coke"},
}

D = list(transactions.values())          # database các giao dịch
N = len(D)                                # tổng số giao dịch
items = sorted({i for t in D for i in t}) # tập item

print("N =", N)
```

```
print("Items =", items)
pd.DataFrame({"Items": [", ".join(sorted(t)) for t in D]},␣
 ↪index=list(transactions)).rename_axis("TID")
```

```
N = 5
Items = ['Beer', 'Bread', 'Coke', 'Diaper', 'Eggs', 'Milk']
```

[2]:

TID	Items
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk

1.2 2. Cài đặt giải thuật Apriori

1.2.1 2.1 Đếm support

$$\text{count}(I) = |\{t \in D : I \subseteq t\}| \qquad s(I) = \frac{\text{count}(I)}{N}$$

[3]:
```
def support_count(D, itemset):
    """Số giao dịch trong D có chứa itemset."""
    return sum(1 for t in D if itemset <= t)
```

1.2.2 2.2 Sinh ứng viên: $F_{k-1} \times F_{k-1}$ + prune

- **Join**: ghép 2 itemset trong F_{k-1} có chung $k-2$ phần tử đầu (sau khi sắp xếp) → ứng viên k item. Cách này không sinh trùng lặp.
- **Prune**: theo *Apriori principle*, nếu một tập con $(k-1)$ item của ứng viên **không** frequent thì ứng viên đó chắc chắn không frequent → loại ngay, khỏi phải quét CSDL.

[4]:
```
def apriori_gen(prev_freq, k):
    """Sinh tập ứng viên C_k từ F_(k-1) bằng phương pháp F_(k-1) x F_(k-1)."""
    prev = sorted(tuple(sorted(s)) for s in prev_freq)
    candidates = []

    # --- Join step ---
    for i in range(len(prev)):
        for j in range(i + 1, len(prev)):
            if prev[i][:k - 2] == prev[j][:k - 2]:       # chung k-2 phần tử đầu
                cand = frozenset(prev[i]) | frozenset(prev[j])
                if len(cand) == k:
                    candidates.append(cand)
            else:
                break                                    # đã sort nên có thể␣
 ↪dừng sớm
```

```
    # --- Prune step (anti-monotone) ---
    prev_set = set(prev_freq)
    pruned = [c for c in candidates
              if all(frozenset(sub) in prev_set
                     for sub in combinations(sorted(c), k - 1))]
    return pruned
```

1.2.3 2.3 Vòng lặp chính của Apriori

```
F_1 = {các item đơn có support >= minSup}
k = 2
while F_(k-1) != rỗng:
    C_k    = apriori_gen(F_(k-1))     # join + prune
    đếm support cho từng ứng viên trong C_k (quét CSDL 1 lần)
    F_k    = {c in C_k : support(c) >= minSup}
    k = k + 1
kết quả = hợp của tất cả F_k
```

[5]:
```
def apriori(D, min_support, verbose=True):
    """Trả về dict {itemset: (support, count)} của mọi tập phổ biến."""
    N = len(D)
    items = sorted({i for t in D for i in t})

    result = {}
    Ck = [frozenset([i]) for i in items]     # C_1
    k = 1

    while Ck:
        # quét CSDL đếm support cho các ứng viên
        Fk = {}
        for c in Ck:
            cnt = support_count(D, c)
            if cnt / N >= min_support:
                Fk[c] = cnt

        if verbose:
            print(f"C_{k}: {len(Ck):>2} ứng viên  ->  F_{k}: {len(Fk):>2} tập␣
 ↪phổ biến")

        if not Fk:
            break

        result.update({s: (c / N, c) for s, c in Fk.items()})
        Ck = apriori_gen(list(Fk), k + 1)     # sinh C_(k+1)
        k += 1

    return result
```

1.2.4 2.4 Sinh luật kết hợp

Với mỗi tập phổ biến I ($|I| \geq 2$), tách I thành X và $Y = I \setminus X$:

$$\text{support}(X \Rightarrow Y) = s(I) \qquad \text{confidence}(X \Rightarrow Y) = \frac{s(I)}{s(X)} \qquad \text{lift}(X \Rightarrow Y) = \frac{\text{confidence}}{s(Y)}$$

Giữ lại luật có `confidence >= minConf`.

[6]:
```python
def generate_rules(freq, min_confidence):
    """Sinh luật X => Y từ các tập phổ biến."""
    sup = {s: v[0] for s, v in freq.items()}
    rules = []

    for I, (s_I, cnt) in freq.items():
        if len(I) < 2:
            continue
        for r in range(1, len(I)):
            for ante in combinations(sorted(I), r):
                X = frozenset(ante)
                Y = I - X
                conf = s_I / sup[X]
                if conf >= min_confidence:
                    rules.append({
                        "X": X, "Y": Y,
                        "support": s_I, "count": cnt,
                        "confidence": conf,
                        "lift": conf / sup[Y],
                    })
    return rules
```

1.3 3. Chọn ngưỡng

CSDL chỉ có $N = 5$ giao dịch nên ngưỡng phải đủ thấp để còn luật, nhưng đủ cao để loại nhiễu:

- **`minSup = 0.4`** itemset phải xuất hiện ít nhất $0.4 \times 5 = 2$ lần. Nếu lấy 0.2 (1 lần) thì mọi itemset xuất hiện dù chỉ 1 lần cũng "phổ biến" → vô nghĩa; nếu lấy 0.6 (3 lần) thì mất hết tập 3 item.
- **`minConf = 0.6`** — ngưỡng thông dụng: luật đúng ít nhất 60% số lần vế trái xuất hiện.
- Tham khảo thêm **`lift > 1`** để loại các luật chỉ "đúng" do item quá phổ biến.

[7]:
```python
MIN_SUP  = 0.4
MIN_CONF = 0.6

print(f"minSup  = {MIN_SUP}  (>= {int(MIN_SUP * N)} giao dịch)")
print(f"minConf = {MIN_CONF}\n")

freq = apriori(D, MIN_SUP)
print(f"\nTổng cộng: {len(freq)} tập phổ biến")
```

```
minSup  = 0.4  (>= 2 giao dịch)
minConf = 0.6

C_1:  6 ứng viên  ->  F_1:  5 tập phổ biến
C_2: 10 ứng viên  ->  F_2:  8 tập phổ biến
C_3:  5 ứng viên  ->  F_3:  4 tập phổ biến

Tổng cộng: 17 tập phổ biến
```

1.4 4. Kết quả — các tập phổ biến

[8]:
```
freq_df = pd.DataFrame([
    {"k": len(s), "itemset": "{" + ", ".join(sorted(s)) + "}", "count": c,␣
 ↪"support": sup}
    for s, (sup, c) in freq.items()
]).sort_values(["k", "support", "itemset"], ascending=[True, False, True]).
 ↪reset_index(drop=True)

freq_df
```

[8]:

	k	itemset	count	support
0	1	{Bread}	4	0.8
1	1	{Diaper}	4	0.8
2	1	{Milk}	4	0.8
3	1	{Beer}	3	0.6
4	1	{Coke}	2	0.4
5	2	{Beer, Diaper}	3	0.6
6	2	{Bread, Diaper}	3	0.6
7	2	{Bread, Milk}	3	0.6
8	2	{Diaper, Milk}	3	0.6
9	2	{Beer, Bread}	2	0.4
10	2	{Beer, Milk}	2	0.4
11	2	{Coke, Diaper}	2	0.4
12	2	{Coke, Milk}	2	0.4
13	3	{Beer, Bread, Diaper}	2	0.4
14	3	{Beer, Diaper, Milk}	2	0.4
15	3	{Bread, Diaper, Milk}	2	0.4
16	3	{Coke, Diaper, Milk}	2	0.4

[9]:
```
# In theo từng mức F_k
for k, g in freq_df.groupby("k"):
    print(f"===== F_{k} ({len(g)} tập) =====")
    for _, r in g.iterrows():
        print(f"  {r['itemset']:<35} count = {r['count']}   support =␣
 ↪{r['support']:.2f}")
    print()
```

```
===== F_1 (5 tập) =====
```

```
  {Bread}                                  count = 4   support = 0.80
  {Diaper}                                 count = 4   support = 0.80
  {Milk}                                   count = 4   support = 0.80
  {Beer}                                   count = 3   support = 0.60
  {Coke}                                   count = 2   support = 0.40

===== F_2 (8 tập) =====
  {Beer, Diaper}                           count = 3   support = 0.60
  {Bread, Diaper}                          count = 3   support = 0.60
  {Bread, Milk}                            count = 3   support = 0.60
  {Diaper, Milk}                           count = 3   support = 0.60
  {Beer, Bread}                            count = 2   support = 0.40
  {Beer, Milk}                             count = 2   support = 0.40
  {Coke, Diaper}                           count = 2   support = 0.40
  {Coke, Milk}                             count = 2   support = 0.40

===== F_3 (4 tập) =====
  {Beer, Bread, Diaper}                    count = 2   support = 0.40
  {Beer, Diaper, Milk}                     count = 2   support = 0.40
  {Bread, Diaper, Milk}                    count = 2   support = 0.40
  {Coke, Diaper, Milk}                     count = 2   support = 0.40
```

[10]:
```python
# Tập phổ biến tối đại (maximal): không có tập cha nào cũng phổ biến
all_sets = set(freq)
maximal = [s for s in all_sets
           if not any(s < other for other in all_sets)]

print("Các tập phổ biến TỐI ĐẠI (maximal frequent itemsets):")
for s in sorted(maximal, key=lambda s: (-len(s), sorted(s))):
    print(f"  {{{', '.join(sorted(s))}}}  support = {freq[s][0]:.2f}")
```

```
Các tập phổ biến TỐI ĐẠI (maximal frequent itemsets):
  {Beer, Bread, Diaper}  support = 0.40
  {Beer, Diaper, Milk}  support = 0.40
  {Bread, Diaper, Milk}  support = 0.40
  {Coke, Diaper, Milk}  support = 0.40
```

1.5 5. Kết quả — các luật kết hợp

[11]:
```python
rules = generate_rules(freq, MIN_CONF)

rules_df = pd.DataFrame([
    {
        "rule": "{" + ", ".join(sorted(r["X"])) + "} => {" + ", ".
 ↪join(sorted(r["Y"])) + "}",
        "support": r["support"],
```

```
        "confidence": r["confidence"],
        "lift": r["lift"],
    }
    for r in rules
]).sort_values(["confidence", "support", "lift"], ascending=False).
 ↪reset_index(drop=True)

print(f"Số luật thoả minConf = {MIN_CONF}: {len(rules_df)}")
rules_df
```

```
Số luật thoả minConf = 0.6: 27
```

[11]:

	rule	support	confidence	lift
0	{Beer} => {Diaper}	0.6	1.000000	1.250000
1	{Coke} => {Diaper, Milk}	0.4	1.000000	1.666667
2	{Coke} => {Diaper}	0.4	1.000000	1.250000
3	{Coke} => {Milk}	0.4	1.000000	1.250000
4	{Beer, Bread} => {Diaper}	0.4	1.000000	1.250000
5	{Beer, Milk} => {Diaper}	0.4	1.000000	1.250000
6	{Coke, Diaper} => {Milk}	0.4	1.000000	1.250000
7	{Coke, Milk} => {Diaper}	0.4	1.000000	1.250000
8	{Diaper} => {Beer}	0.6	0.750000	1.250000
9	{Bread} => {Diaper}	0.6	0.750000	0.937500
10	{Diaper} => {Bread}	0.6	0.750000	0.937500
11	{Bread} => {Milk}	0.6	0.750000	0.937500
12	{Milk} => {Bread}	0.6	0.750000	0.937500
13	{Diaper} => {Milk}	0.6	0.750000	0.937500
14	{Milk} => {Diaper}	0.6	0.750000	0.937500
15	{Diaper, Milk} => {Coke}	0.4	0.666667	1.666667
16	{Beer} => {Bread, Diaper}	0.4	0.666667	1.111111
17	{Bread, Diaper} => {Beer}	0.4	0.666667	1.111111
18	{Beer} => {Diaper, Milk}	0.4	0.666667	1.111111
19	{Diaper, Milk} => {Beer}	0.4	0.666667	1.111111
20	{Beer} => {Bread}	0.4	0.666667	0.833333
21	{Beer} => {Milk}	0.4	0.666667	0.833333
22	{Beer, Diaper} => {Bread}	0.4	0.666667	0.833333
23	{Beer, Diaper} => {Milk}	0.4	0.666667	0.833333
24	{Bread, Diaper} => {Milk}	0.4	0.666667	0.833333
25	{Bread, Milk} => {Diaper}	0.4	0.666667	0.833333
26	{Diaper, Milk} => {Bread}	0.4	0.666667	0.833333

[12]:
```
# In dạng văn bản
print(f"{'LUẬT':<45}{'SUPPORT':>9}{'CONFIDENCE':>13}{'LIFT':>8}")
print("-" * 75)
for _, r in rules_df.iterrows():
    print(f"{r['rule']:<45}{r['support']:>9.2f}{r['confidence']:>13.
 ↪2f}{r['lift']:>8.2f}")
```

```
LUẬT                                                SUPPORT   CONFIDENCE    LIFT
-------------------------------------------------------------------------------
{Beer} => {Diaper}                                     0.60         1.00    1.25
{Coke} => {Diaper, Milk}                               0.40         1.00    1.67
{Coke} => {Diaper}                                     0.40         1.00    1.25
{Coke} => {Milk}                                       0.40         1.00    1.25
{Beer, Bread} => {Diaper}                              0.40         1.00    1.25
{Beer, Milk} => {Diaper}                               0.40         1.00    1.25
{Coke, Diaper} => {Milk}                               0.40         1.00    1.25
{Coke, Milk} => {Diaper}                               0.40         1.00    1.25
{Diaper} => {Beer}                                     0.60         0.75    1.25
{Bread} => {Diaper}                                    0.60         0.75    0.94
{Diaper} => {Bread}                                    0.60         0.75    0.94
{Bread} => {Milk}                                      0.60         0.75    0.94
{Milk} => {Bread}                                      0.60         0.75    0.94
{Diaper} => {Milk}                                     0.60         0.75    0.94
{Milk} => {Diaper}                                     0.60         0.75    0.94
{Diaper, Milk} => {Coke}                               0.40         0.67    1.67
{Beer} => {Bread, Diaper}                              0.40         0.67    1.11
{Bread, Diaper} => {Beer}                              0.40         0.67    1.11
{Beer} => {Diaper, Milk}                               0.40         0.67    1.11
{Diaper, Milk} => {Beer}                               0.40         0.67    1.11
{Beer} => {Bread}                                      0.40         0.67    0.83
{Beer} => {Milk}                                       0.40         0.67    0.83
{Beer, Diaper} => {Bread}                              0.40         0.67    0.83
{Beer, Diaper} => {Milk}                               0.40         0.67    0.83
{Bread, Diaper} => {Milk}                              0.40         0.67    0.83
{Bread, Milk} => {Diaper}                              0.40         0.67    0.83
{Diaper, Milk} => {Bread}                              0.40         0.67    0.83
```

[13]:
```python
# Các luật đáng quan tâm nhất: confidence = 1.0 và lift > 1
best = rules_df[(rules_df["confidence"] >= 1.0) & (rules_df["lift"] > 1)]
print("Luật mạnh nhất (confidence = 100%, lift > 1):")
best.reset_index(drop=True)
```

```
Luật mạnh nhất (confidence = 100%, lift > 1):
```

[13]:

	rule	support	confidence	lift
0	{Beer} => {Diaper}	0.6	1.0	1.250000
1	{Coke} => {Diaper, Milk}	0.4	1.0	1.666667
2	{Coke} => {Diaper}	0.4	1.0	1.250000
3	{Coke} => {Milk}	0.4	1.0	1.250000
4	{Beer, Bread} => {Diaper}	0.4	1.0	1.250000
5	{Beer, Milk} => {Diaper}	0.4	1.0	1.250000
6	{Coke, Diaper} => {Milk}	0.4	1.0	1.250000
7	{Coke, Milk} => {Diaper}	0.4	1.0	1.250000

1.6 6. Kiểm chứng bằng package FIM (PyFIM — Christian Borgelt)

So sánh kết quả tự cài đặt với package `fim` (http://www.borgelt.net) để chắc chắn phần cài đặt đúng.

Lưu ý: `fim.apriori` nhận `supp/conf` theo **phần trăm**, và mặc định chỉ sinh luật có **vế phải 1 item**.

[14]:
```
import fim

tracts = [sorted(t) for t in D]

fim_freq  = fim.apriori(tracts, target='s', supp=MIN_SUP * 100, zmin=1,␣
 ↪report='aS')
fim_rules = fim.apriori(tracts, target='r', supp=MIN_SUP * 100, conf=MIN_CONF *␣
 ↪100,
                         zmin=2, report='aSC')

# bản tự cài đặt: lọc các luật có vế phải 1 item cho đồng nhất với PyFIM
my_single = [r for r in rules if len(r["Y"]) == 1]

print(f"Tự cài đặt : {len(freq):>2} tập phổ biến, {len(my_single):>2} luật (vế␣
 ↪phải 1 item)")
print(f"PyFIM      : {len(fim_freq):>2} tập phổ biến, {len(fim_rules):>2} luật")

same_sets  = set(freq) == {frozenset(s) for s, cnt, pct in fim_freq}
same_rules = ({(r["X"], r["Y"]) for r in my_single}
              == {(frozenset(body), frozenset([head])) for head, body, *_ in␣
 ↪fim_rules})

print("\nTrùng khớp tập phổ biến :", same_sets)
print("Trùng khớp luật kết hợp :", same_rules)
```

```
Tự cài đặt : 17 tập phổ biến, 24 luật (vế phải 1 item)
PyFIM      : 17 tập phổ biến, 24 luật

Trùng khớp tập phổ biến : True
Trùng khớp luật kết hợp : True
```

1.7 7. Nhận xét

- Với `minSup = 0.4` và `minConf = 0.6`, thuật toán dừng sau F_3: bước join+prune từ F_3 không sinh được ứng viên 4 item nào, nên $C_4 = \emptyset$.
- Bước prune phát huy tác dụng rõ: ở mức $k = 3$ số ứng viên bị cắt giảm đáng kể so với việc sinh mọi tổ hợp $\binom{6}{3} = 20$.
- Các luật có `confidence = 1.0`, ví dụ `{Beer} => {Diaper}` (support 0.6): mọi khách mua Beer đều mua Diaper — đây chính là luật "bia – tã" kinh điển trong Market-Basket Analysis.
- `{Coke} => {Diaper, Milk}` có `lift = 1.67` (cao nhất), nghĩa là mối liên hệ này mạnh hơn hẳn mức ngẫu nhiên.

- Kết quả tự cài đặt trùng khớp 100% với package `fim` (PyFIM).